



ADMAYA.AI

AI Provider Integration Playbook

Connecting Seedance, Veo, Kling, Higgsfield, HeyGen, Runway & aggregators to the backend

A hand-off document for your developer

Prepared for Buddy Tezz — August 2026

1. The Short Version

Your web app never talks to Higgsfield, Seedance, Kling, HeyGen, Veo or Runway directly from the browser. Your backend does — it's the only place that ever holds an API key. When a user hits “Generate”, your backend merges the locked template with their script, calls the right provider's API, waits for the render, downloads the result, and only then shows it to the user. Every provider below is a separate account, a separate API key, and (usually) a separate cost — your developer's main job is building one clean internal layer that can call any of them the same way.

2. API vs. MCP — Which Does Your Developer Actually Build?

You asked about “API, MCP and all” specifically, so this is worth answering directly: your product is built on REST APIs, not MCP. Here's the difference and why it matters.

- A **REST API** is a web address your server calls with an API key, sends a request, gets a job ID back, other software for years, and it's built to once. Every provider in this document offers



Edit with the Docs app

Make tweaks, leave comments, and share with others to edit at the same time.

NO THANKS

GET THE APP

fferent job: letting an AI agent (like Claude, e, on behalf of the person typing. Higgsfield, ii) that exposes 30+ models — but by their P,” i.e. a chat assistant, not a website's

is worth knowing about only if you separately want an internal admin tool — e.g. ask Claude to generate 10 sample videos for a new template” —

a convenience for your own team, entirely outside the product your users touch. The diagram below keeps those two paths visually separate for exactly this reason.



Production request path (what your developer builds) versus the agent-only MCP path (an optional internal convenience, not part of the product).

3. The Universal Integration Pattern

Every provider below follows the same five-step shape, because video rendering takes anywhere from 30 seconds to several minutes — nothing here is a request that returns an answer instantly.

- **1. Store the API key server-side only.** Environment variable or secrets manager — never in any code that reaches the browser.
- **2. Build the payload.** Merge the template's locked prompt (environment, camera, style — see the template JSON schema in the business plan doc) with the user's script.
- **3. Submit the job.** The provider returns a job ID immediately and starts rendering in the background. Runway's API, for example, moves a job through QUEUED → PROCESSING (with a progress percentage) → COMPLETED or FAILED — a pattern that's near-universal across this whole category.
- **4. Find out when it's done.** Either poll the status endpoint every few seconds until it says COMPLETED, or — where the provider supports it — register a webhook URL they call back automatically. Webhooks are more efficient; polling is the reliable fallback every provider supports.
- **5. Download and re-host the result.** Save the finished video to your own storage (S3, Cloud Storage, etc.) immediately. Provider-hosted result URLs are usually temporary and can expire — don't link to them long-term.

```
// Generic shape every adapter implements
interface ProviderAdapter {
  submitJob(payload: RenderPayload): Promise<{ jobId: string }>;
  checkStatus(jobId: string): Promise<'QUEUED'|'PROCESSING'|'COMPLETED'|'FAILED'>;
  getResult(jobId: string): Promise<{ videoUrl: string; durationSec: number }>;
}
```

4. Provider-by-Provider Reference

Pricing below is indicative, pulled from public sources as of August 2026 — video-generation API pricing moves often, so re-check each provider's own pricing page before committing budget.

4.1 Seedance (ByteDance)

Best for: fast, cheap UGC-style product/ad clips — a strong default for most of your template library.

- **Access:** official route is BytePlus; also available through aggregators fal.ai, PiAPI, WaveSpeed, Kie AI, SeeGen AI — all pay-as-you-go, no subscription.
- **Cost:** roughly \$0.23–\$0.47 per second of 720p video depending on route — BytePlus is the cheapest/official option, aggregators cost a bit more for easier multi-model integration.
- **Async pattern:** submit → poll or webhook → download, same as everyone else here.

4.2 Kling (Kuaishou)

Best for: strong general-purpose video quality, a common alternative/backup to Seedance.

- **Access:** official Kling AI Open Platform for developers, or through the same aggregators as Seedance if you want one API key for both.
- **Cost:** varies by model version, resolution and whether native audio is generated — priced per second in their newer models. Get current numbers from the Open Platform pricing page before committing, since tiers change.

4.3 Higgsfield

Best for: cinematic camera movement and production value — good for higher-end brand templates, not for high-volume batch UGC.

- **Access:** official cloud API at cloud.higgsfield.ai, with a Python SDK (higgsfield-client). Consumer subscriptions (\$15–\$99/mo) are separate from API usage, which is billed on its own, usage-based.
- **Also offers:** a public MCP server (see section 2) — useful for your team, not for the product backend.

4.4 HeyGen

Best for: avatar talking-head templates — a presenter speaking the user's script direct to camera (your “Founder Talks Straight” and “Customer Review” style templates).

- **Access:** official developer API, separate from the consumer web app, credit-metered.
- **Cost:** scales sharply with avatar quality tier — their highest-realism avatar tier can cost several times more per minute than the base tier, so price your templates per the specific tier you actually use, not the headline plan price.

4.5 Google Veo (via Vertex AI)

Best for: premium, high-fidelity shots where quality matters more than cost — not a fit for your everyday low-cost templates.

- **Access:** the official programmatic path is Google Cloud Vertex AI — create a GCP project, enable the Vertex AI API, and authenticate with a service account. Setup takes hours, not minutes, since it's full Google Cloud infrastructure.
- **Cost:** metered per second — roughly \$0.75/second, so an 8-second clip runs about \$6. That's meaningfully more expensive than Seedance or Runway's cheaper tiers, so reserve Veo for a small number of premium/flagship templates rather than your default engine.
- **Note on “Gemini”:** Gemini is Google's language model, not the video engine — it's genuinely useful in this stack for a different job: polishing a user's rough script, checking it against your content-moderation rules, or auto-writing the three ad-variant hooks in your “Product → Ad” flow. Veo is the part that turns a script into video.

4.6 Runway

Best for: stylized, cinematic clips at a genuinely low cost — worth testing as a default engine, not just a niche option.

- **Access:** no waitlist — create an account at dev.runwayml.com, set up an organization for billing, add a minimum \$10 prepaid balance, and generate an API key (RUNWAYML_API_SECRET).
- **Cost:** Gen-4 Turbo runs about \$0.05/second (a 5-second clip is roughly \$0.25) — one of the cheapest options in this whole table. Gen-4.5 (higher quality) runs about \$0.12/second.
- **Async pattern:** textbook — QUEUED → PROCESSING → COMPLETED/FAILED, both polling and webhooks supported, and failed jobs auto-refund credits on their side.

4.7 D-ID

An alternative to HeyGen for avatar/talking-head video — worth a side-by-side quality and price test before committing to one avatar provider, since the two are close competitors.

4.8 Aggregators — fal.ai, Replicate, PiAPI, WaveSpeed, Kie AI

These sit in front of many of the providers above (Seedance, Kling, and others) behind a single API key and a consistent request shape. They typically cost a little more per generation than going direct, but cut integration work from “one new integration per provider” to “one integration, many models” — genuinely useful for getting an MVP running fast, with the option to move a specific high-volume template to a direct/official integration later once its economics matter.

5. Matching Providers to Your Templates

Template	Category	Suggested engine	Why
Unbox & React	E-commerce UGC	Seedance	Cheap, fast, good for high-volume product content
Before / After Transform	Beauty & wellness	Higgsfield	Camera/cinematic quality matters for the reveal
App Walkthrough	App / SaaS demo	Screen-record + Seedance overlay	Mostly compositing, not full generative video
Founder Talks Straight	D2C founder story	HeyGen	Needs a consistent, realistic talking presenter
Customer Review, UGC	Testimonial	HeyGen or D-ID	Avatar-led, conversational delivery
Feature Spotlight	App / SaaS highlight	Runway	Cheap, stylized motion graphics-style clips
Limited-Time Offer	E-commerce promo	Runway	Low cost fits high-frequency promo content
Street Interview Reaction	Awareness	Kling or Seedance	Naturalistic motion, handheld

Treat this as a starting point, not a rule — the template JSON's provider field (see the business plan doc, section 4.1) makes swapping engines a one-field change once you've tested real output quality against real cost.

6. The Abstraction Layer — What Your Developer Actually Builds

One interface, many adapters. Nothing else in your codebase should know which provider is behind a given template — it only calls the abstraction layer.

```
// providers/seedanceAdapter.ts (via an aggregator, e.g. fal.ai)
export const SeedanceAdapter: ProviderAdapter = {
  async submitJob(payload) {
    const res = await fetch('https://fal.run/fal-ai/seedance', {
      method: 'POST',
      headers: { Authorization: `Key ${process.env.FAL_KEY}` },
      body: JSON.stringify(buildSeedancePrompt(payload)),
    });
    const { request_id } = await res.json();
    return { jobId: request_id };
  },
  async checkStatus(jobId) { /* GET status endpoint, map to enum */ },
  async getResult(jobId) { /* GET result, return hosted video URL */ },
};

// providers/runwayAdapter.ts
export const RunwayAdapter: ProviderAdapter = {
  async submitJob(payload) {
    const res = await fetch('https://api.dev.runwayml.com/v1/generate', {
      method: 'POST',
      headers: { Authorization: `Bearer ${process.env.RUNWAYML_API_SECRET}` },
      body: JSON.stringify(buildRunwayPrompt(payload)),
    });
    const { id } = await res.json();
    return { jobId: id };
  },
  async checkStatus(jobId) { /* GET job → QUEUED/PROCESSING/COMPLETED/FAILED */ },
  async getResult(jobId) { /* GET job → output video URL */ },
};

// orchestrator.ts
const ADAPTERS = { seedance: SeedanceAdapter, runway: RunwayAdapter, /* ...others */ };
const adapter = ADAPTERS[template.provider]; // one field on the template picks the engine
```

7. Webhooks vs. Polling — Practical Notes

- **Prefer webhooks where a provider supports them** (Runway does) — register your endpoint once, and the provider notifies you the moment a render finishes instead of you asking repeatedly.
- **Poll as the universal fallback:** every provider here supports a plain status check. A reasonable default is polling every 3–5 seconds with a sane timeout (e.g. 10 minutes) — if a job hasn't completed by then, mark it failed and auto-refund the user's credits rather than leaving them waiting indefinitely.
- **Log every state transition** (submitted → processing → completed/failed) against the job — this is what makes the credit ledger and margin reporting in the business plan doc actually trustworthy.

8. Secrets, Rate Limits & Cost Tracking — a Checklist

- **One API key per provider**, stored in your backend's secrets manager (not .env files committed to git, not client-side code).
- **Rate-limit per user**, not just globally — ties directly into the wallet system, since a user's credit balance is already a natural throttle.
- **Record the real provider cost against every job**, not just what you charged the user — this is the only way to know if a template's margin is actually holding up once you're live.
- **Set a monthly spend alert** with each provider (most support this natively) so a bug that loops on generation doesn't turn into a surprise bill.

9. Suggested Build Order

- **Week 1–2:** integrate one aggregator (fal.ai or PiAPI) covering Seedance — this alone can power most of your launch templates and gets a working end-to-end pipeline (template → script → video → storage) proven fastest.
- **Week 3–4:** add Runway direct (cheap, clean API, good for the promo/feature-style templates) and HeyGen (for avatar-led templates).
- **Week 5–6:** add Higgsfield for your premium/cinematic templates; evaluate Veo on a couple of flagship templates only, given its per-second cost.
- **Ongoing:** keep the abstraction layer clean enough that swapping a template's engine — say, moving a cost-sensitive template from Higgsfield to Seedance — is a one-field config change, not a rebuild.

10. Quick-Reference Cheat Sheet

Provider	Access	Approx. cost	Best for
Seedance	BytePlus (official) or fal.ai / PiAPI / WaveSpeed	\$0.23–\$0.47/sec	Default — cheap, fast UGC/product clips
Kling	Kling AI Open Platform, or aggregators	Varies by tier — check live pricing	General-purpose alternative to Seedance
Higgsfield	cloud.higgsfield.ai + higgsfield-client SDK	Usage-based, separate from \$15–\$99/mo plans	Cinematic, premium-look templates
HeyGen	Official developer API	Credit-metered, scales with avatar tier	Avatar talking-head templates
Google Veo	Vertex AI (GCP project + service account)	≈\$0.75/sec (≈\$6 for 8s)	Small number of flagship/premium templates
Runway	dev.runwayml.com, prepaid balance	\$0.05–\$0.12/sec	Cheap, stylized clips; clean webhook support
D-ID	Official API	Credit-metered	Alternative avatar engine to HeyGen
fal.ai / PiAPI / etc.	One API key, many models	Small premium over direct	Fastest path to an MVP

Hand this document and the business plan's technical section (template JSON schema, wallet/ledger design) to your developer together — between the two, they have the product model, the money model, and the exact list of accounts to open.